Event-Driven Multi-Image FPGA Reconfiguration: RTL Architecture and a Preliminary Verification Study

Anonymous working manuscript

ABSTRACT

This paper describes the RTL architecture and a preliminary verification study of an autonomous controller that sequences configuration of multiple FPGA images from external serial flash. The platform combines a metadata-driven scheduler, AXI4-Lite control writes, a memory-to-JTAG streaming path, and sticky interrupt handling for target task events. The scheduler advances on configuration and task-completion events, while a runtime limit serves as a safety watchdog. A focused Vivado XSim testbench exercises programmer package definitions, interrupt capture and clearing, and a scheduler AXI-write failure followed by abort handling. The supplied simulation transcript reports 94 package value checks, 12 interrupt scenarios, three scheduler scenarios, and 29 scoreboard checks, completing at 390 ns without a testbench failure. An integrated three-image testbench is also present in the repository and specifies checks for metadata, QSPI transactions, stream beats, JTAG activity, and task-driven phase changes. The available XCRG attempt did not find a functional coverage database; therefore, this study reports directed checks rather than coverage percentages.

Index Terms—FPGA configuration, SystemVerilog, event-driven scheduling, AXI4-Stream, JTAG, RTL verification.

I. INTRODUCTION

FPGAs can be configured through vendor-supported interfaces including JTAG and configuration-memory paths. A multi-image controller must coordinate image metadata, memory reads, streaming transfer, target identification, configuration status, and application-level completion. These responsibilities cross several interfaces and include failure paths that are difficult to assess from an isolated controller test. Vendor documentation defines device configuration behavior, while AMBA specifications define the control and streaming protocols; application-specific sequencing and recovery remain system design concerns [1], [2].

Prior FPGA scheduling research has explored dynamic partial reconfiguration and task preemption, whereas remote JTAG work has studied extending programming paths over optical links [4], [5]. The design studied here takes a different, narrower scope: a full-image, metadata-driven control path that advances between images when the configured workload reports completion. It does not implement partial reconfiguration, task preemption, or a general-purpose operating system scheduler.

This paper makes three bounded contributions. First, it documents an RTL decomposition for autonomous multi-image configuration using a scheduler, control-register hub, memory manager, DMA stream bridge, JTAG controller, and interrupt hub. Second, it describes a self-checking integrated testbench architecture for a three-image QSPI-to-JTAG flow. Third, it reports the results of a focused XSim unit test and distinguishes those checks from unavailable XCRG coverage data. The reported evidence is preliminary: it is simulation-only and does not include FPGA-board measurements, synthesis utilization, or timing closure.

II. PLATFORM ARCHITECTURE

The top-level design connects a time-multiplex scheduler to an AXI4-Lite register bank. Register writes configure the memory manager, DMA controller, and JTAG controller. The selected memory path in the integrated simulation is QSPI; SPI and OSPI controller modules are also present in the RTL. The memory manager returns metadata and image bytes over a stream interface. During metadata acquisition, the scheduler consumes the stream directly; image data is routed through the DMA bridge to the JTAG controller.

Fig. 1. Control and configuration data paths in the RTL platform.

CONTROL	Scheduler → AXI4-Lite register hub → memory / DMA / JTAG
DATA	QSPI NOR → memory manager → DMA stream bridge → JTAG → target TAP
FEEDBACK	Target configuration status and task events → synchronizers / IRQ hub → scheduler
PACKAGE	programmer_pkg supplies shared parameters, encodings, packed types, and CRC helper

The scheduler reads a metadata table containing one four-word record per image: flash offset, bitstream length in bits, expected IDCODE, and maximum task duration. It writes configuration values through a single-outstanding AXI4-Lite write master, then waits for the memory, DMA, and JTAG engines to complete. It requests target configuration status before enabling the fabric. After release, a sticky target-task-done interrupt advances the image phase; a task-error interrupt or watchdog expiry leads to the abort sequence. Durations are runtime limits, not fixed phase timers.

The DMA controller is a single-entry AXI4-Stream bridge. It checks nonzero byte-aligned transfer lengths, contiguous byte enables, expected final-beat framing, and the declared bit count. A registered output beat provides one-entry backpressure decoupling. The JTAG controller drives the target TAP and supports IDCODE capture, instruction scans, data shifts, and abort recovery. These interfaces follow the protocol roles of AXI4-Lite, AXI4-Stream, and IEEE 1149.1 JTAG [2], [3].

A. Configuration phases and control contracts

The metadata path is a bootstrap phase. At schedule start, the controller configures the memory interface to fetch the metadata region and starts a read for twelve 32-bit words, corresponding to three image records. A record supplies the flash address, transfer size in bits, expected target IDCODE, and duration limit. The RTL validates the metadata count and fields before using the records. This separates discovery of the image set from the later configuration of each image. If the metadata stream is incomplete, malformed, or times out, the scheduler selects the abort sequence rather than releasing the target fabric.

For each image, the scheduler writes memory, DMA, and JTAG register values before starting device identification. The JTAG sequence selects the IDCODE instruction, captures the target identifier, compares it with the image record, and then selects the configuration instruction. Only after the stream transfer and successful target status does the scheduler assert target_fabric_start_en. This ordering encodes a safety boundary in the control FSM: data movement and target release are distinct phases, and a mismatch or unsuccessful configuration status enters abort handling.

The scheduler's AXI4-Lite write master permits address and data channels to handshake independently and consumes the response only after both are accepted. A write error redirects the sequence to three control writes that abort memory, DMA, and JTAG. Recovery is bounded by an abort-wait watchdog and JTAG abort completion. The interrupt controller registers the enable mask, captures rising edges into sticky status bits, derives pending bits from status and enable, and clears status through an explicit request. The top-level design synchronizes target-originated task and configuration status signals before they enter the synchronous control domain.

Normal phase changes are event-driven. After configuration, the target runs until its task-done interrupt is observed. The scheduler then disables the target fabric, acknowledges the sticky event, and

either advances the phase index or marks the schedule complete. A task-error interrupt disables the fabric and begins abort recovery. The duration field is implemented as a maximum-runtime guard; it is not a periodic timer used to switch images. This distinction is important because the testbench varies task runtimes to check event-based phase advancement.

III. VERIFICATION METHOD

A. Integrated testbench structure

The repository contains a procedural SystemVerilog top-level testbench, a behavioral multi-image QSPI NOR model, and a behavioral JTAG target model. The testbench initializes three distinct nine-byte images and twelve metadata words, starts the scheduler, and models target configuration status and task-completion events. It checks metadata transfer counts, QSPI opcodes and addresses, memory-to-DMA and DMA-to-JTAG beat contents, JTAG TDI bits, IDCODE and instruction selection, status handshakes, task timing, interrupt acknowledgement, and final scheduler status. The design uses a 10 MHz simulation clock and requests three images with distinct task runtimes of 45, 120, and 15 microseconds.

Each image contains 72 bits. The stream scoreboard expects two full four-byte beats followed by a one-byte final beat with TKEEP set to 0001 and TLAST asserted. The model observes the QSPI read opcode and address sequence, compares each memory and DMA beat with the expected image words, and samples TDI on the target TCK edge during DR shifting. Separate monitor logic checks that IDCODE and configuration instructions have been committed before corresponding operations, and that target configuration status is accepted only after the TAP has returned to Run-Test/Idle.

TABLE II. CHECK FAMILIES ENCODED IN THE INTEGRATED TESTBENCH

Boundary	Testbench check	Expected nominal evidence
Metadata	Count and order	12 words; three image records
QSPI	Opcode and address log	One metadata plus three image reads
Streaming	Data, TKEEP, TLAST	3 beats per image; final 1-byte beat
JTAG	Instruction, IDCODE, TDI	12 completion events; 72 bits per image
Scheduling	Status and task events	Three config handshakes and three done events

The monitor checks that the target fabric is not enabled before a successful configuration-status handshake, that the compatibility timer output tracks fabric enable, that the target task watchdog does not expire during the nominal workload, and that the target completion/error IRQ bits are acknowledged. At completion, the testbench compares the phase index and aggregate counts against its directed scenario expectations. Together, these checks provide explicit cross-component observations rather than relying only on the scheduler's finished flag.

The integrated testbench is a directed nominal-flow test, not a UVM environment. Its monitors and scoreboard use procedural checks that call $fatal on mismatches. The repository snapshot does not contain the UVM driver, sequence, scoreboard, or four-test regression suite described in the supplied project report. Accordingly, UVM and those error-injection results are not treated as executed evidence here.

B. Focused module test

A separate testbench instantiates programmer_pkg-dependent checks, the interrupt controller, and the scheduler with an AXI-Lite response model. Package checks cover the declared memory, trigger, command, response, error, interrupt, DMA, controller-state, and TAP-state encodings, as well as the CRC update function. Interrupt scenarios exercise reset, registered enable configuration, rising-edge capture, sticky status, masking, clearing, held-high sources, simultaneous sources, and clear priority. The scheduler test injects an AXI write error, services the subsequent abort writes, and checks that the controller reaches failure with the target fabric disabled.

This focused test is also directed. Its totals are scoreboard counters and scenario checks; they are not covergroup hit counts. Although SystemVerilog covergroups were added to the testbench after the supplied passing transcript, the transcript used for this paper predates that change. The subsequent XCRG invocation reported that no functional coverage database was present. Therefore, no XCRG percentage or bin-closure result is reported.

The package checks include categorical value comparisons for the parameterized design constants and enumerated protocol/control values, plus two CRC input checks. The interrupt scenarios use a 16-source instance and distinguish stored status from enabled pending status. In particular, the test checks that a held-high source does not repeatedly relatch after clear, that a new low-to-high transition does, and that clear priority is respected when a new event is presented on the same cycle. These checks target behavior that can be missed by merely observing irq_active at the end of a test.

C. Requirement-to-check traceability

The focused bench follows a small requirement-to-check map. The table below identifies the behavior intended by each group of checks and the evidence actually available from the supplied run. It is not a statement of design-wide coverage. For example, checking a clear request on a particular interrupt bit does not imply that every source bit, every enable-mask permutation, or every simultaneous clear/set combination has been explored.

TABLE III. FOCUSED REQUIREMENTS AND EVIDENCE BOUNDARIES

Requirement	Directed check	Evidence boundary
Package definitions	Constants, enum values, CRC helper	Specified package checks only
Interrupt edge capture	Enabled low-to-high source event	Selected bits and sequences
Sticky status	Hold, clear, mask, and re-arm	Directed scenarios; not exhaustive
IRQ concurrency	Multiple sources and clear priority	One crafted simultaneous case
Scheduler recovery	AXI write error, abort writes, terminal fail	One injected write-error path

The scheduler case uses a minimal bus responder rather than the complete peripheral stack. It accepts the first requested AXI-Lite write, returns a nonzero response, and then acknowledges the three abort writes. The test checks the resulting failed status, disabled target fabric, inactive timer indication, and unchanged phase. This isolates the scheduler's control response to a register-write failure, but it does not test errors originating independently in the memory, DMA, JTAG, or target configuration paths.

The interrupt test uses the RTL's registered enable behavior explicitly. A source edge is only expected to latch when the prior cycle's registered enable is set. This explains why enable programming and source-edge generation are separate testbench steps. After clear, a held-high source should not immediately create another edge; a low interval followed by a new rise should. The case set therefore checks both level persistence and edge re-arming rather than treating irq_src as a continuously asserted request.

IV. PRELIMINARY RESULTS

The supplied Vivado XSim transcript for the focused testbench shows successful compilation and elaboration, followed by completion at 390 ns without a testbench failure. The testbench printed the following directed-check totals. These counts describe its checks and do not estimate the fraction of RTL behavior exercised.

TABLE I. FOCUSED XSIM TESTBENCH COUNTS REPORTED BY THE SUPPLIED RUN

Target	Reported checks	Interpretation
programmer_pkg	94	Package value/function checks
interrupt_controller	12	Directed scenarios

time_multiplex_scheduler	3	Directed scenarios
Combined scoreboard	29	Individual pass/fail checks
Simulation	390 ns	Reached $finish; no TB failure

The integrated testbench source defines stronger end-to-end nominal checks: twelve metadata handshakes, four QSPI reads (metadata plus three images), three configuration-status handshakes, three task-done events, twelve JTAG completion events, sixty-four AXI-Lite writes, and seventy-two shifted image bits per phase. These values are testbench expectations encoded as pass/fail checks. The repository snapshot includes the testbench but not a corresponding successful XSim transcript, so this paper does not present them as measured run results. A project report separately records a 655.75 microsecond nominal completion, but the underlying run log and UVM sources are not present in the checked-in snapshot; that timing is omitted from the measured results.

V. DISCUSSION AND LIMITATIONS

The focused run demonstrates that the package checks, interrupt edge and sticky-status scenarios, and one scheduler write-error recovery path complete under the supplied XSim setup. It is useful as a fast unit-level regression, while the integrated testbench provides a concrete structure for checking interactions across flash, stream, JTAG, and target-event boundaries. The two levels should remain distinct: the focused run does not establish end-to-end operation, and the integrated testbench source alone does not establish a passing simulation.

Several evidence gaps remain before this work is ready for a full research submission. The XCRG report could not be generated because the supplied run produced no functional-coverage database. The repository contains no UVM sources or automated four-test regression script, and the DMA RTL still uses $countones() rather than the iterative keep_to_bits() fix described in the project report. No synthesis, resource-utilization, maximum-frequency, physical-board, or fault-injection measurements are available in this snapshot. These items should be reconciled and rerun before making coverage-closure, portability, or hardware-readiness claims.

TABLE IV. RECONCILIATION OF PROJECT-REPORT CLAIMS WITH THIS SOURCE SNAPSHOT

Claim or artifact	Checked-in evidence	Treatment in this paper
UVM-1.2 / four-test regression	No UVM files or runner found	Not reported as executed
FSM and transition coverage	No scheduler covergroup in integrated TB	No coverage percentage claimed
XCRG report	Supplied xcrg log reports no database	Marked unavailable
DMA keep_to_bits fix	RTL still calls $countones()	Not credited as applied

A stronger evaluation should archive exact source revisions and simulator logs; rerun the integrated nominal test and each negative test; generate an XCRG database from the final source; report the covergroup model and bin definitions; and add synthesis and timing data for a named FPGA part. If the intended contribution is the documented simulator-portability fix or UVM-based fault-recovery methodology, those source changes and regression artifacts should be checked into the same revision used for submission.

VI. CONCLUSION

This paper documents an RTL platform for event-driven sequencing of multiple full FPGA images and a preliminary layered verification approach. The architecture combines metadata parsing, AXI-Lite configuration, QSPI-backed streaming, DMA flow control, JTAG operations, and sticky target-event interrupts. The available XSim evidence is a focused directed test that reports 94 package checks, 12 interrupt scenarios, three scheduler scenarios, and 29 scoreboard checks, completing at 390 ns. Because the XCRG database and full-regression artifacts are unavailable, the results are presented as preliminary checks rather than coverage closure. The next step is to make the source, regression, and coverage database mutually reproducible before evaluating this design as a conference-ready verification methodology.

REFERENCES

[1] AMD, 7 Series FPGAs Configuration User Guide, UG470, v1.17, Dec. 2023.

[2] Arm, AMBA 4 AXI4 and AXI4-Stream Protocol Specifications, Arm Ltd.

[3] IEEE, IEEE Standard for Test Access Port and Boundary-Scan Architecture, IEEE Std 1149.1-2013, May 2013.

[4] G. Rodriguez-Canal, N. Brown, Y. Torres, and A. Gonzalez-Escribano, "Task-based preemptive scheduling on FPGAs leveraging partial reconfiguration," arXiv:2301.07615, 2023.

[5] B. Deng et al., "JTAG-based Remote Configuration of FPGAs over Optical Fibers," arXiv:2401.15746, 2024.

[6] Project RTL and testbench sources: programmer_pkg.sv; interrupt_controller.sv; time_multiplex_scheduler.sv; axi_lite_write_master.sv; fpga_multiphase_reconfig_top.sv; tb_multi_image_schedule.sv; tb_rtl_coverage.sv. Project source snapshot, Nov. 2026.